TRAVAUX PRATIQUES · OPENSTACK COMPUTE & IMAGE

LAB03

Compute & Images : Nova et Glance

Module « Infrastructures cloud natives » · 2026-2027

CONTENU DU DOCUMENT

00 Objectifs
P0 Partie 0 — Vérification de l'environnement OpenStack
P1 Partie 1 — Glance : gestion des images
P2 Partie 2 — Les gabarits (flavors)
P3 Partie 3 — Personnalisation au démarrage avec cloud-init
P4 Partie 4 — Cycle de vie d'une instance
P5 Partie 5 — Snapshots d'instance
P6 Partie 6 — Côté hyperviseur : ce que Nova a créé
→ Nettoyage

MODULE
Infrastructures cloud natives

PLATEFORME
OpenStack · Nova · Glance

ANNÉE
2026-2027

00 Objectifs

- Gérer des images avec **Glance** : import, formats, propriétés, visibilité et partage entre projets ;
- Définir des **gabarits (flavors)** adaptés à un besoin ;
- Personnaliser une instance au démarrage avec **cloud-init**;
- Piloter le **cycle de vie** d'une instance Nova (stop, pause, suspend, shelve, resize) et expliquer l'effet de chaque opération ;
- Créer un **snapshot d'instance** et redéployer à partir de celui-ci ;
- Observer côté hyperviseur (**libvirt / KVM**) ce que Nova a réellement créé.

Partie 0 Vérification de l'environnement OpenStack

Si l'instance EC2 a été arrêtée depuis la dernière séance, vérifiez que les services OpenStack sont bien repartis.

TERMINAL

```
sudo systemctl list-units 'devstack@*' --no-pager
source ~/devstack/openrc admin admin      # adaptez le chemin si nécessaire
openstack compute service list
openstack hypervisor list
openstack network agent list
```

Tous les services doivent être `up` / `enabled`. Si un service est arrêté :

TERMINAL

```
sudo systemctl restart devstack@<service>
```

Vérifiez ensuite le bridge externe `br-ex`, qui relie l'hôte au réseau `public` (172.24.4.0/24) des IP flottantes :

TERMINAL

```
ip addr show br-ex
```

Le bridge doit être `UP` et porter l'adresse `172.24.4.1/24`. DevStack configure cette adresse lors de l'installation, mais elle **n'est pas conservée après un redémarrage** de l'instance EC2. Si elle est absente, rétablissez-la :

TERMINAL

```
sudo ip addr add 172.24.4.1/24 dev br-ex
sudo ip link set br-ex up
```

> **Q1**
> Quels services Nova apparaissent dans `openstack compute service list` ? Expliquez en une phrase le rôle de chacun (`nova-scheduler` , `nova-conductor` , `nova-compute`).

Partie 1 Glance : gestion des images

1.1 Explorer les images existantes

TERMINAL

```
openstack image list
openstack image show <id-image-cirros>
```

> **Q2**
> Relevez le format disque (`disk_format`), le format conteneur (`container_format`), la taille, la visibilité et le propriétaire de l'image CirrOS.

1.2 Importer une image Ubuntu cloud

TERMINAL

```
wget https://cloud-images.ubuntu.com/minimal/releases/noble/release/ubuntu-24.04-minimal-cloudimg-amd64.img
qemu-img info ubuntu-24.04-minimal-cloudimg-amd64.img
```

> **Q3**
> Quel est le format réel du fichier selon `qemu-img info` ? Comparez sa taille sur disque (*disk size*) et sa taille virtuelle (*virtual size*). Pourquoi ces deux valeurs diffèrent-elles ?

Créez l'image dans Glance :

TERMINAL

```
openstack image create ubuntu-24.04-minimal \
    --file ubuntu-24.04-minimal-cloudimg-amd64.img \
    --disk-format qcow2 \
    --container-format bare \
    --min-disk 4 \
    --min-ram 512 \
    --property os_distro=ubuntu \
    --property os_version=24.04 \
    --private
```

Vérifiez où Glance a stocké l'image (backend `file` par défaut dans DevStack) :

TERMINAL

```
openstack image show ubuntu-24.04-minimal -c id -c status -c checksum
sudo ls -lh /opt/stack/data/glance/images/
```

> **Q4**
> Quel est le nom du fichier stocké par Glance ? Quel lien a-t-il avec l'image ? À quoi servent `--min-disk` et `--min-ram` ?

1.3 Visibilité et partage entre projets

Glance propose quatre niveaux de visibilité : `private` , `shared` , `community` , `public` .

a) Créer un projet et un utilisateur de test

Avec les identifiants `admin` :

TERMINAL

```
source ~/devstack/openrc admin admin
openstack project create project-lab03
openstack user create student-lab03 --project project-lab03 --password Lab03Pass
openstack role add --project project-lab03 --user student-lab03 member
```

> **Important :** un administrateur voit toutes les images, quelle que soit leur visibilité. Pour observer réellement l'effet du partage, il faut se placer du point de vue d'un utilisateur **non administrateur**.

b) Vérifier que l'image est invisible pour le nouveau projet

Ouvrez un **second terminal** sur l'instance EC2 et connectez-vous en tant que `student-lab03` :

TERMINAL

```
source ~/devstack/openrc admin admin
export OS_USERNAME=student-lab03
export OS_PASSWORD=Lab03Pass
export OS_PROJECT_NAME=project-lab03
openstack token issue -c project_id -c user_id
openstack image list
```

c) Partager l'image avec le projet

Dans le **premier terminal** (admin) :

TERMINAL

```
openstack image set --shared ubuntu-24.04-minimal
openstack image add project ubuntu-24.04-minimal project-lab03
openstack image member list ubuntu-24.04-minimal
```

d) Accepter le partage

Dans le **second terminal** (`student-lab03`) :

TERMINAL

```
openstack image list
openstack image set --accept <id-image-ubuntu>
openstack image list
```

Vérifiez à nouveau le statut du membre dans le premier terminal :

TERMINAL

```
openstack image member list ubuntu-24.04-minimal
```

> **Q5**
> L'image est-elle visible pour `student-lab03` **avant** le partage ? Après le partage mais **avant** l'acceptation ? Après l'acceptation ? Quel est le statut du membre (`pending` , `accepted` , `rejected`) à chaque étape ?

> **Q6**
> Quelle différence entre une image `shared` et une image `community` ? Qui peut rendre une image `public` et pourquoi cette opération est-elle réservée à ce rôle ?

Partie 2 Les gabarits (flavors)

TERMINAL

```
openstack flavor list
```

Créez deux gabarits (avec les identifiants `admin`) :

TERMINAL

```
openstack flavor create lab03.small  --vcpus 1 --ram 1024 --disk 5
openstack flavor create lab03.medium --vcpus 2 --ram 2048 --disk 8
```

Créez un gabarit privé, réservé au projet du Lab 03 (`project-lab03`) :

TERMINAL

```
openstack flavor create lab03.private --vcpus 1 --ram 512 --disk 4 --private
openstack flavor set lab03.private --project <project-lab03>
```

> **Q7**
> Peut-on lancer l'image Ubuntu avec le gabarit `m1.tiny` ? Testez puis expliquez le message obtenu en lien avec `--min-disk` et `--min-ram`.

> **Q8**
> Le gabarit `lab03.private` apparaît-il dans `openstack flavor list` lorsqu'on est connecté avec un autre projet ? À quoi sert un gabarit privé dans un cloud d'entreprise ?

Partie 3 Personnalisation au démarrage avec cloud-init

Toutes les commandes de cette partie sont exécutées sur l'instance EC2, avec les identifiants `admin` (premier terminal).

Placez-vous dans votre répertoire personnel : la clé et le fichier user-data y seront créés.

TERMINAL

```
cd ~
```

3.1 Préparer le réseau et l'accès à l'instance

a) Réseau privé, sous-réseau et routeur

TERMINAL

```
openstack network create net-lab03
openstack subnet create subnet-lab03 \
    --network net-lab03 \
    --subnet-range 192.168.30.0/24 \
    --dns-nameserver 8.8.8.8
openstack router create router-lab03
openstack router set router-lab03 --external-gateway public
openstack router add subnet router-lab03 subnet-lab03
```

b) Groupe de sécurité

Autorisez SSH, HTTP et le ping :

TERMINAL

```
openstack security group create sg-lab03 --description "SSH, HTTP et ICMP pour le Lab 03"
openstack security group rule create --protocol tcp --dst-port 22 sg-lab03
openstack security group rule create --protocol tcp --dst-port 80 sg-lab03
openstack security group rule create --protocol icmp sg-lab03
openstack security group rule list sg-lab03
```

c) Paire de clés

TERMINAL

```
openstack keypair create cle-lab03 > cle-lab03.pem
chmod 600 cle-lab03.pem
```

3.2 Écrire le fichier user-data

Le fichier doit être créé dans le même répertoire que la clé (votre répertoire personnel), car la commande `openstack server create` de la section 3.3 y fera référence :

TERMINAL

```
cd ~
nano web-init.yaml
```

Collez le contenu suivant, puis enregistrez et quittez :

WEB-INIT.YAML (CLOUD-CONFIG)

```
#cloud-config
hostname: vm-web
write_files:
  - path: /var/www/index.html
    content: |
      <h1>Lab 03 - Instance configuree par cloud-init</h1>
runcmd:
  - cd /var/www && nohup python3 -m http.server 80 > /var/log/web.log 2>&1 &
```

> **Attention à l'indentation :** le YAML est sensible aux espaces. Utilisez des espaces, jamais de tabulations.

Signification des directives :

Directive	Rôle
`#cloud-config`	Première ligne **obligatoire** : indique à cloud-init que le fichier est au format cloud-config (YAML)
`hostname`	Nom d'hôte attribué à la VM
`write_files`	Crée des fichiers dans la VM (ici la page d'accueil du site)
`runcmd`	Commandes exécutées en `root` à la fin du premier démarrage (ici, lancement d'un serveur web Python sur le port 80)

Vérifiez le fichier :

TERMINAL

```
ls -l web-init.yaml
cat web-init.yaml
sudo cloud-init schema --config-file web-init.yaml
```

La dernière commande valide la syntaxe du fichier avec l'outil cloud-init installé sur l'instance EC2. Le résultat attendu est `Valid schema web-init.yaml`.

> **Sécurité :** le contenu du user-data reste lisible par tout processus de la VM pendant toute sa durée de vie. N'y placez jamais de mot de passe, de clé privée ou de jeton d'accès.

3.3 Lancer l'instance

a) Créer l'instance

TERMINAL

```
openstack server create vm-web \
    --image ubuntu-24.04-minimal \
    --flavor lab03.small \
    --network net-lab03 \
    --security-group sg-lab03 \
    --key-name cle-lab03 \
    --user-data web-init.yaml \
    --wait
```

L'option `--wait` rend la main une fois l'instance `ACTIVE`. À ce stade, elle ne possède qu'une adresse **privée** (`192.168.30.x`).

b) Associer une IP flottante

L'IP flottante est une ressource Neutron distincte de l'instance : on la réserve dans le réseau `public`, puis on l'associe à la VM. L'adresse est enregistrée dans la variable `FIP`, réutilisée dans la suite du lab :

TERMINAL

```
FIP=$(openstack floating ip create public -f value -c floating_ip_address)
openstack server add floating ip vm-web $FIP
echo $FIP
```

c) Suivre le démarrage

Le premier démarrage peut prendre plusieurs minutes. Relancez la commande suivante jusqu'à voir la ligne `Cloud-init v. ... finished` :

TERMINAL

```
openstack console log show vm-web | grep -i cloud-init
```

d) Tester le service web

Si ce n'est pas déjà fait (Partie 0), activez le bridge externe `br-ex`, qui relie l'hôte au réseau des IP flottantes :

TERMINAL

```
sudo ip addr add 172.24.4.1/24 dev br-ex
sudo ip link set br-ex up
```

Définissez les fonctions `web` et `relance_web`, qui testent et relancent le serveur web :

TERMINAL

```
web() {
  curl -sS -m 5 http://$FIP || echo "=> pas de réponse du serveur web"
}

relance_web() {
  ssh -n -i ~/cle-lab03.pem \
      -o StrictHostKeyChecking=no -o ConnectTimeout=5 \
      ubuntu@$FIP \
      "cd /var/www && sudo nohup python3 -m http.server 80 > /tmp/web.log 2>&1 < /dev/null &" \
    && echo "=> serveur web relancé"
}
```

Vérifiez la connectivité et le service :

TERMINAL

```
ping -c 3 $FIP
web
```

Le résultat attendu est la page `<h1>Lab 03 - Instance configuree par cloud-init</h1>`.

Diagnostic : si `web` ne répond pas, le message d'erreur de `curl` indique où chercher :

Erreur	Signification	Action
Refus immédiat (*Failed to connect ... after 0 ms*)	La VM répond, mais aucun service n'écoute sur le port 80	Relancer le serveur : `relance_web` puis `web`
Délai dépassé (*timed out*) ou ping sans réponse	Problème réseau, ou VM pas encore démarrée	Vérifier `br-ex` (Partie 0), attendre la fin du démarrage (console), puis réessayer

Partie 4 Cycle de vie d'une instance

Pour chaque opération ci-dessous, relevez le **statut** Nova, le `vm_state`, le `power_state`, et l'état vu par l'hyperviseur.

Toutes les commandes sont exécutées sur l'instance EC2, avec les identifiants `admin`, dans le **même terminal qu'en Partie 3** (la variable `FIP` et les fonctions `web` et `relance_web` y sont déjà définies).

Pour simplifier les observations, définissez une petite fonction `etat` dans votre terminal :

TERMINAL

```
etat() {
  openstack server show vm-web -f shell \
    -c status -c flavor -c OS-EXT-STS:vm_state -c OS-EXT-STS:power_state -c OS-EXT-STS:task_state
  sudo virsh list --all
}
etat
```

> **Important :** les commandes de cycle de vie sont **asynchrones**. Elles rendent la main immédiatement, alors que l'opération se poursuit en arrière-plan (`task_state` différent de `None`). Lancez donc **une seule opération à la fois**, relancez `etat` jusqu'à ce que le `task_state` revienne à `None` , relevez les valeurs, **puis** passez à l'opération suivante. Enchaîner deux commandes (par exemple `stop` puis `start` sur la même ligne) provoque une erreur `409 Conflict` , car l'instance est encore en cours de transition.
> **Nouveau terminal ?** Les fonctions et la variable `FIP` n'existent que dans le terminal où elles ont été définies. Dans un nouveau terminal, redéfinissez `web` , `relance_web` et `etat` , et récupérez l'adresse flottante :
> `FIP=$(openstack floating ip list --port $(openstack port list --server vm-web -f value -c ID) -f value -c "Floating IP Address")`

Après chaque opération **inverse** (`start` , `unpause` , `resume` , `unshelve` , `reboot`), testez le serveur web avec `web` . S'il ne répond pas, notez-le pour la Q9, puis relancez-le avec `relance_web` avant de passer à l'opération suivante.

a) Arrêt et redémarrage

TERMINAL

```
openstack server stop vm-web
etat                                  # attendre : SHUTOFF, task_state None
openstack server start vm-web
etat                                  # attendre : ACTIVE
web
```

b) Pause

TERMINAL

```
openstack server pause vm-web
etat
openstack server unpause vm-web
etat
web
```

c) Suspension

TERMINAL

```
openstack server suspend vm-web
etat
openstack server resume vm-web
etat
web
```

d) Mise en réserve (shelve)

La commande `openstack server shelve` met une instance « sur l'étagère » : l'instance est arrêtée puis **retirée du serveur de calcul**, afin de libérer les ressources (vCPU, RAM, disque local) qu'elle y occupait. Elle continue pourtant d'exister pour OpenStack : elle reste visible dans `openstack server list` et conserve son identité (UUID, nom, adresses IP, paire de clés). La commande inverse, `openstack server unshelve`, la fait revenir sur un serveur de calcul et la redémarre.

C'est l'opération adaptée à une instance qu'on n'utilisera pas pendant longtemps, mais qu'on souhaite retrouver telle quelle plus tard. Observez attentivement ce qui change dans Nova, dans libvirt **et** dans Glance.

TERMINAL

```
openstack server shelve vm-web
etat                                  # attendre : SHELVED_OFFLOADED
openstack image list                  # une nouvelle image est apparue
openstack server unshelve vm-web
etat                                  # attendre : ACTIVE
openstack image list
web
```

e) Redémarrages

TERMINAL

```
openstack server reboot --soft vm-web
etat
openstack server reboot --hard vm-web
etat
web
```

> **Q9**
> Complétez le tableau suivant :

Opération	Statut Nova	État libvirt	Où se trouve l'état mémoire ?	Ressources (CPU/RAM) libérées sur l'hôte ?	Serveur web actif après l'opération inverse ?
stop					
pause					
suspend					
shelve					

> **Q10**
> Après `shelve`, une nouvelle image apparaît dans Glance. Pourquoi ? Que se passe-t-il au moment du `unshelve` ?

> **Q11**
> Différence entre `reboot --soft` et `reboot --hard` ?

4.1 Redimensionnement (resize)

Redimensionnez l'instance vers le gabarit `lab03.medium`, puis **annulez** l'opération :

TERMINAL

```
openstack server resize --flavor lab03.medium vm-web
etat                                  # attendre : VERIFY_RESIZE, task_state None
openstack server resize revert vm-web
etat                                  # attendre : ACTIVE — quel est le gabarit ?
```

Recommencez, puis **confirmez** cette fois :

TERMINAL

```
openstack server resize --flavor lab03.medium vm-web
etat                                  # attendre : VERIFY_RESIZE, task_state None
openstack server resize confirm vm-web
etat                                  # ACTIVE avec lab03.medium
```

Essayez enfin de redimensionner vers un gabarit dont le disque est **plus petit** :

TERMINAL

```
openstack server resize --flavor m1.tiny vm-web
openstack image show ubuntu-24.04-minimal -c min_disk
```

> **Q12**
> Quel est le statut de l'instance juste après le resize, avant confirmation ? Que permettent `openstack server resize revert` et `openstack server resize confirm` ?

> **Q13**
> Que se passe-t-il lors du redimensionnement vers `m1.tiny` et pourquoi ? Comparez la taille minimale indiquée dans le message d'erreur avec le `min_disk` de l'image Ubuntu : pourquoi ces deux valeurs diffèrent-elles ?

4.2 Historique des actions

TERMINAL

```
openstack server event list vm-web
openstack server event show vm-web <request-id>      # choisissez le request-id du resize
```

> **Q14**
> À quoi sert le `request-id` ? En quoi cet historique est-il utile en exploitation ?

Avant de passer à la Partie 5, assurez-vous que le serveur web répond :

TERMINAL

```
relance_web
web
```

Partie 5 Snapshots d'instance

Connectez-vous à la VM et ajoutez une ligne à la page d'accueil :

TERMINAL

```
ssh -i ~/cle-lab03.pem ubuntu@$FIP
```

Dans la VM :

TERMINAL

```
echo "<p>Modifie apres le demarrage</p>" | sudo tee -a /var/www/index.html
exit
```

De retour sur l'instance EC2, vérifiez la modification, puis prenez un snapshot :

TERMINAL

```
web
openstack server image create vm-web --name snap-vm-web --wait
openstack image show snap-vm-web -c status -c size -c min_disk -c properties
```

Lancez une nouvelle instance à partir du snapshot, **sans user-data**, et associez-lui une IP flottante :

TERMINAL

```
openstack server create vm-web-clone \
    --image snap-vm-web \
    --flavor lab03.medium \
    --network net-lab03 \
    --security-group sg-lab03 \
    --key-name cle-lab03 \
    --wait

FIP_CLONE=$(openstack floating ip create public -f value -c floating_ip_address)
openstack server add floating ip vm-web-clone $FIP_CLONE
echo $FIP_CLONE
```

Attendez la fin du démarrage (`openstack console log show vm-web-clone | tail -5`), puis testez le clone :

TERMINAL

```
ping -c 3 $FIP_CLONE
curl -sS -m 5 http://$FIP_CLONE
ssh -i ~/cle-lab03.pem ubuntu@$FIP_CLONE "cat /var/www/index.html"
```

> **Q15**
> Le clone contient-il la modification faite à la main ? Le serveur web est-il démarré automatiquement ? Expliquez.

Partie 6 Côté hyperviseur : ce que Nova a créé

Jusqu'ici, vous avez manipulé les instances à travers les commandes `openstack`, c'est-à-dire la vue logique d'OpenStack. Dans cette partie, vous allez observer ce qui existe réellement sur l'hôte de calcul : la machine virtuelle définie dans **libvirt** et ses fichiers disque.

Toutes les commandes sont exécutées sur l'instance EC2, avec les identifiants `admin`.

6.1 La machine virtuelle dans libvirt

Récupérez l'UUID de l'instance et son nom libvirt :

TERMINAL

```
ID=$(openstack server show vm-web -f value -c id)
INST=$(openstack server show vm-web -f value -c OS-EXT-SRV-ATTR:instance_name)
echo "UUID : $ID"
echo "Nom libvirt : $INST"
```

Affichez les caractéristiques de la VM, puis la description de son disque dans la configuration XML générée par Nova :

TERMINAL

```
sudo virsh dominfo $INST
sudo virsh dumpxml $INST | grep -E "<vcpu|<memory"
sudo virsh dumpxml $INST | grep -A6 "<disk"
```

6.2 Le disque de l'instance

Listez les fichiers de l'instance et examinez son disque :

TERMINAL

```
sudo ls -lh /opt/stack/data/nova/instances/$ID/
sudo qemu-img info /opt/stack/data/nova/instances/$ID/disk
```

Repérez la ligne `backing file`, puis examinez le cache des images de base :

TERMINAL

```
sudo ls -lh /opt/stack/data/nova/instances/_base/
```

6.3 Comparaison avec le clone

Faites la même observation pour `vm-web-clone` :

TERMINAL

```
ID_CLONE=$(openstack server show vm-web-clone -f value -c id)
sudo qemu-img info /opt/stack/data/nova/instances/$ID_CLONE/disk
```

Comparez les champs `backing file`, `virtual size` et `disk size` des deux instances.

> **Q16**
> Retrouvez dans le XML libvirt le nombre de vCPU et la quantité de mémoire. Correspondent-ils au gabarit actuel de l'instance ?

Nettoyage

Supprimez les ressources propres à ce lab, mais **conservez celles qui serviront au Lab 04**.

a) Supprimer le clone, son IP flottante et le snapshot

TERMINAL

```
openstack server delete vm-web-clone --wait
openstack floating ip delete $FIP_CLONE
openstack image delete snap-vm-web
```

> Supprimer une instance **ne libère pas** son IP flottante : elle reste réservée dans le projet tant qu'elle n'est pas supprimée explicitement.

b) Mettre `vm-web` de côté

L'instance `vm-web` sera réutilisée au Lab 04 (attachement d'un volume Cinder). Mettez-la en réserve pour libérer les ressources de l'hôte entre les deux séances :

TERMINAL

```
openstack server shelve vm-web
```

c) Vérifier qu'il ne reste pas de ressources orphelines

TERMINAL

```
openstack server list
openstack image list
openstack floating ip list          # aucune IP avec un champ Port à "None"
```

Si une IP flottante n'est associée à aucun port, supprimez-la :

TERMINAL

```
openstack floating ip delete <adresse>
```

d) Ressources conservées pour le Lab 04

Ressource	Nom
Instance (en réserve)	`vm-web` et son IP flottante
Image	`ubuntu-24.04-minimal`
Gabarits	`lab03.small`, `lab03.medium`, `lab03.private`
Réseau	`net-lab03`, `subnet-lab03`, `router-lab03`
Groupe de sécurité	`sg-lab03`
Paire de clés	`cle-lab03` (et le fichier `~/cle-lab03.pem`)
Projet et utilisateur	`project-lab03`, `student-lab03`